

Project Report Structure

The project report should provide a complete and well-structured overview of your work. It must clearly explain the main objectives of the project, the technologies used, and how the system was designed and implemented. Each section should describe your process — from initial planning and design to development, testing, and deployment.

In addition, the report must include a section that explains the specific responsibilities and contributions of each team member. Every student should describe the part they worked on (for example, backend service, frontend interface, or API gateway), the challenges they faced, and the solutions they applied.

The report should also contain supporting materials such as diagrams, screenshots, code snippets, or test results where appropriate. Finally, you are expected to conclude the report with a reflection on what you learned during the project, including both technical and teamwork skills.

- ✓ Use **simple language** and **bullet points** instead of long paragraphs.
- ✓ Keep **code examples small** (only a few important lines).
- ✓ Add **one small diagram** instead of many.
- ✓ Use **tables** for students' responsibilities and test results (saves space).
- ✓ Avoid repeating the same explanations in each section

1. Cover Page

- Project title
 - Group number / Team name
 - Names of all 3 students (with IDs)
 - Supervisor name
 - Date of submission
-

2. Abstract

A short summary (around 100–150 words) explaining:

- What the project is about
- Which technologies/languages were used
- Main goal and outcome

3. Introduction

- Explain what microservices are in simple terms
- Why using multiple languages is useful
- The aim of your project (for example: an online shop, student management, weather info, etc.)
- What the system can do

4. System Overview

- Show the **system idea** (for example: “Our system lets users order products online.”)
- Describe the **microservices** (each one written in a different language)
- Explain how they talk to each other (using APIs or messages)

5. System Architecture

- Include a **flowchart or diagram** showing:
 - The services
 - Their connections
 - Which part uses which programming language

6. Implementation (Main Coding Section)

Explain in detail how you built each part:

Service 1 (Language 1 – e.g., Python)

- Function or role (e.g., handles user login)
- Tools/libraries used
- Code explanation (brief)

Service 2 (Language 2 – e.g., JavaScript/Node.js)

- Function or role (e.g., manages product catalog)
- Tools/libraries used
- Code explanation (brief)

Service 3 (Language 3 – e.g., Java or PHP)

- Function or role (e.g., payment or order processing)
 - Tools/libraries used
 - Code explanation (brief)
-

7. Communication between Services

- How they exchange data (API calls, HTTP requests, JSON files, etc.)
 - Example of how one service sends or receives data
-

8. User Interface

- Describe what users can see and do (web or mobile app)
 - Include screenshots
-

9. Testing

- Explain how you tested each service
 - Mention some example test results
 - Note any problems you faced and how you fixed them
-

10. Results

- Final working output
 - What your system can successfully do
 - Show any screenshots or sample outputs
-

11. Discussion

- What went well
- What was difficult (for example: connecting languages, debugging, etc.)
- How teamwork helped

✓ *All students contribute one paragraph.*

12. Conclusion and Future Work

- Summary of what you achieved
 - What improvements can be done in future (e.g., adding new features or using databases)
-

13. References

- List of tutorials, documentation, or websites you used.

Section	Description	Suggested Pages
1. Cover Page	Title, names, date, etc.	1 page
2. Abstract	Short summary	0.5 page
3. Introduction	What and why of the project	1 page
4. System Overview	Simple explanation of the system idea	1 page
5. System Architecture	One diagram + short explanation	1 page
6. Implementation	3 short subsections (one per language)	3–4 pages
7. Communication Between Services	How the microservices interact	1 page
8. User Interface	Screenshots + short text	1–2 pages
9. Testing & Results	Describe tests + final output	1.5 pages
10. Discussion	Problems faced + teamwork	1 page
11. Conclusion & Future Work	Summary & suggestions	0.5 page
12. References	Links, code snippets	1–2 pages